

Let's Write a Simple JPEG Library, Part-II: The Decoder

Mar 2, 2019 • Koushtav Chakrabarty

Part-I

This is part two of three in the series of articles that I'm writing about the JPEG specification. Part one came out more than one year ago and I had several other commitments (namely I got a job now!) absorbing my time before I could turn my attention back to this unfinished series. Initially, I wanted part two to be the encoder, but then decided that implementing a decoder first would be more easier (in my honest opinion). And once you know how to write the decoder, the encoder will be the exact same steps, but in the reverse order!

I already have the functional code for the working decoder available on [GitHub](#). Feel free to consult it before, during or after reading the article!

Now, without further ado, let's get on with writing a JPEG decoder!

Table of Contents

1. Recap of JPEG Compression
 - What is JPEG?
 - Colour Model Used by JPEG
 - Discrete Cosine Transform and its Relevance in JPEG
 - Quantization and its Purpose
 - Entropy Coding Using Run-Length Encoding and Huffman Coding
2. What is a Decoder?
3. The JFIF File Format
 - Detailed Description of the Markers
 - Start of Image Segment (SOI)
 - JPEG/JFIF Image Segment (APP-0)
 - Comment Segment (COM)
 - Define Quantization Table Segment (DQT)
 - Start of Frame-0 Segment (SOF-0)
 - Define Huffman Table Segment (DHT)
 - Start of Scan Segment (SOS)
 - End of Image Segment (EOI)
4. Implementation

- [Setting Up the Project](#)
- [Adding Common Structures and Types](#)
 - [Utility](#)
 - [Transform](#)
 - [Types](#)
 - [Huffman Tree](#)
 - `Node`
 - `HuffmanTree`
 - [Markers](#)
 - [MCU](#)
 - [Image](#)
 - [Decoder](#)
- [The interface for the Decoder](#)
- [Implementing the Decoder](#)
 - `parseAPP0Segment`
 - `parseCOMSegment`
 - `parseDQTSegment`
 - `parseSOF0Segment`
 - `parseDHTSegment`
 - `parseSOSSegment`
 - `scanImageData`
 - `byteStuffScanData`
 - `decodeScanData`
- 5. [Putting it all Together](#)
- 6. [Summary](#)
- 7. [References](#)

Recap of JPEG Compression

In the [previous post](#), we went through the steps involved in JPEG compression. I recommend reading it first if you're not familiar with how JPEG works before proceeding further.

Here's a brief recap of what we covered:

What is JPEG?

- JPEG is an ISO standardized image compression algorithm. It's official standard specifies a well defined set of steps to compress and decompress raw images using JPEG.
- JFIF is the file format used for sharing JPEG compressed images.

Colour Model Used by JPEG

- JPEG uses the YC_bC_r colour model because in this model a pixel can be expressed as the luminance (Y) and chroma (C_b, C_r). Doing so enables JPEG to perform chroma subsampling (throwing away some of the chroma

info) because human eyes are good at discerning brightness than it is for colours.

Discrete Cosine Transform and its Relevance in JPEG

- JPEG operates on blocks of 8x8 pixels, called minimum coded units (MCUs), rather than on the individual pixels.
- The values of the pixels in the MCUs are centered according to the range of permissible intensities for that component.
- DCT is then applied on the MCUs to convert them to their spatial frequencies (which basically means for each MCU, DCT determines what percentage of what frequency can be used to compose the MCU).

Quantization and its Purpose

- The MCU, after it has been transformed using DCT, is quantized so that negligible info about colours and high frequency variations can be thrown away.

Entropy Coding Using Run-Length Encoding and Huffman Coding

- After quantization, similar frequencies are grouped together by performing zig-zag traversal of the MCUs.
- Run length encoding is performed on the vector of values.
- The run-length encoded vector is encoded using Huffman coding to further compress it (DC and AC coefficients are encoded separately).

What is a Decoder?

The steps listed above and covered earlier describe a process that's known as **encoding**. This is because a stream of raw, uncompressed bytes were converted to coded format, using the steps mentioned in the JPEG standard. The reverse of this encoding process is **decoding**, where an encoded piece of data is restored back to its original state by undoing the steps performed while encoding.

So, a JPEG decoder is a program that understands:

- **The JFIF file format:** As observed previously, JPEG is just the image compression algorithm. The encoded data is stored and distributed using the JFIF file standard as prescribed in [ITU-T.871](#).
- **The JPEG compression algorithm:** After reading the encoded data from the JFIF file, it has to replay the steps of encoding in the reverse order to get the original image data back

JPEG decoders are all around us - everytime you use your favourite image viewing tool to open a JPEG file, or browse

social media looking at memes - the application in question is silently using a JPEG library's decoder to decompress the original image data for your perusal.

Before we can start writing our decoder, let's take a look at JFIF.

The JFIF File Format

The JFIF file format specifies **markers** that correspond to different **segments**. Each segment holds a part of the information needed to decode the image. Markers are nothing but words of size two bytes that always start with the byte `ff` and end with a byte that has a value other than `ff` or `00`.

The list of markers and segments that concern us are listed below:

Marker	Segment
<code>ff d8</code>	Start of Image segment (SOI)
<code>ff e0</code>	JPEG/JFIF Image segment (APP-0)
<code>ff fe</code>	Comment segment (COM)
<code>ff db</code>	Define Quantization Table segment (DQT)
<code>ff c0</code>	Start of Frame-0 (SOF-0)
<code>ff c4</code>	Define Huffman Table segment (DHT)
<code>ff da</code>	Start of Scan segment (SOS)
<code>ff d9</code>	End of Image segment (EOI)

There is some restriction in the ordering as observed:

1. SOI
2. APP-0
3. SOS
4. EOI

Other markers like `COM`, `DQT`, etc, may appear in any order but they must be present between the `APP-0` and `SOS` markers.

Additionally, some markers like `APP-0`, `COM`, etc. are followed by additional data immediately after it, whereas markers like `SOI` and `EOI` are self contained.

```
ff xx l1 l2 <payload>
```

Here, the first two bytes `ff` and `xx` describe the marker, followed by two bytes `l1` and `l2` describing the length of the payload following the marker. This is followed by the actual

payload data which is exactly as long as the specified by bytes

11 and 12 .

Our decoder will only deal with images that have the SOF-0 segment define, as SOF-0 corresponds to baseline DCT JPEG images.

Detailed description of the markers

Start of Image Segment (SOI)

- Marker: `ff d8`
- Description: Every JFIF file begins with this marker

JPEG/JFIF Image Segment (APP-0)

Bytes	Size (in bytes)	Description
<code>ff e0</code>	2	APP-0 segment marker
<code><byte></code> <code><byte></code>	2	Length of payload
<code>4a 46 49</code> <code>46 00</code>	5	The string "JFIF\0"
<code><byte></code> <code><byte></code>	2	JFIF version, first byte for major version, second byte for minor version
<code><byte></code>	1	Pixel unit density: <code>00</code> for no units, <code>01</code> for pixels per inch, <code>02</code> for pixels per cm
<code><byte></code> <code><byte></code>	2	Horizontal pixel density, > 0
<code><byte></code> <code><byte></code>	2	Vertical pixel density, > 0
<code><byte></code>	1	Horizontal pixel count of the image thumbnail (0 means no thumbnail)
<code><byte></code>	1	Vertical pixel count of the image thumbnail (0 means no thumbnail)
<code><thumbnail-data></code>	3 × H. pixel count × V. pixel count	Uncompressed 24-bit RGB thumbnail data

(Table borrowed from Wikipedia: [JFIF APP0 marker segment](#))

Comment Segment (COM)

Bytes	Size (in bytes)	Description
ff fe	2	Comment segment marker
<byte> <byte>	2	Length of payload
<comment-data>	n	Actual comment data, n = length of comment as specified in length

Define Quantization Table Segment (DQT)

Bytes	Size (in bytes)	Description
ff db	2	DQT segment marker
<byte> <byte>	2	Length of length of segment, n (including segment marker for DQT). A single DQT segment may have one or more tables
<byte>	1	, first four bits represent precision (0 = 8-bit, 1 = 16-bit), , last four bits represent quantization table #
<quantization-table-data>	64	the data for the quantization table, vector of 64 elements in zig-zag order

NOTE:

The length (n) of the segment specifies the total length of one or more quantization tables within the same marker. So, after quantization table data, more data may follow which contains info for the next quantization table

Start of Frame-0 Segment (SOF-0)

Bytes	Size (in bytes)	Description
ff c0	2	Start of Frame-0 segment marker
<byte> <byte>	2	Length of the segment
<byte>	1	Precision of the frame data
<byte> <byte>	2	Image height in pixels
<byte> <byte>	2	Image width in pixels
<byte>	1	# of components (n) in frame, 3 for RGB images
<byte> <byte> <byte>	$3 \times n$	First byte identifies the component, Second byte represents sampling factor (first four MSBs represent horizontal, last four LSBs represent vertical), Third byte represents which quantization table to use for this component

Define Huffman Table Segment (DHT)

Bytes	Size (in bytes)	Description
ff c4	2	Define Huffman Table segment marker
<byte> <byte>	2	Length of the segment including marker
<byte>	1	Huffman table info, first four MSBs represent table type (0 for DC, 1 for AC), last four LSBs represent table #
<symbol-count>	16	The symbol count (n) for each symbol length from 1-bit to 16-bits
<symbols>	n	The symbols themselves

Start of Scan Segment (SOS)

Bytes	Size (in bytes)	Description
<code>ff da</code>	2	Start of Scan segment marker
<code><byte></code> <code><byte></code>	2	Length of the segment
<code><byte></code>	1	Component count (n)
<code><component-data></code>	$2 \times n$	Component data, first byte denotes component ID, second byte denotes the Huffman table used (first four MSBs denote Huffman table for DC, and last four LSBs denote Huffman table for AC)
<code><skip-bytes></code>	3	Bytes to skip mandatorily

NOTE:

The JPEG compressed image data starts immediately after the SOS segment.

End of Image Segment (EOI)

- Marker: `ff d9`
- Description: Every JFIF file ends with this marker

NOTE:

The EOI segment occurs immediately after the JPEG compressed image data.

A Note About Markers:

This description of JFIF file markers is by no means exhaustive, but is good enough for writing a decoder. I encourage you to take a look at the JFIF specification for details about other markers and info that I haven't covered.

Implementation

Now that we know that goes inside a JFIF file, i.e., the very same `*.jpg` or `*.jpeg` files that get created by applications

like GIMP, or cameras on your Android phone, we can start writing a simple decoder!

NOTE

As mentioned before, I will be using easy to understand, but highly inefficient code during implementation because learning is the main focus rather than producing a production ready JPEG decoder! If you feel that some (or all) parts could be reimplemented without losing readability, please feel free to drop a message with your suggestions! 😊

Setting Up the Project

We will follow a simple and flat project hierarchy for this project. Navigate to an ideal place on your machine and create the following project hierarchy with empty files:

```
libKPEG/  
  include/  
    Decoder.hpp  
    HuffmanTree.hpp  
    Image.hpp  
    Markers.hpp  
    MCU.hpp  
    Transform.hpp  
    Types.hpp  
    Utility.hpp  
  src/  
    Decoder.cpp  
    HuffmanTree.cpp  
    Image.cpp  
    MCU.cpp  
    Transform.cpp  
    Utility.cpp  
  CMakeLists.txt  
  main.cpp
```

You are free to use any text editor, like GVim, or use a fully fledged IDE, like KDevelop (I used KDevelop back when I wrote the decoder). We will also use CMake as our build system.

As you can see, I've named our JPEG library `libKPEG`, but you're free to pick your own name! I've also added some C++ header and source files with self-descriptive names that roughly gives an idea about what piece of functionality goes where. We will be discussing the classes in the next section, so all will be clear soon! Then there's a CMake build script which we will be filling up next to ensure our code is built properly.

```

#
# libKPEG - Simple JPEG Decoder
# =====
#
# CMake build script

cmake_minimum_required(VERSION 3.0)
project(kpeg)

if(NOT CMAKE_BUILD_TYPE)
    set(CMAKE_BUILD_TYPE "Release" CACHE STRING
        "Choose the type of build, options are: Debug Release."
        FORCE)
endif(NOT CMAKE_BUILD_TYPE)

if(CMAKE_COMPILER_IS_GNUCXX OR "${CMAKE_CXX_COMPILER_ID}" STREQUAL
    "Clang")
    set(CMAKE_CXX_FLAGS_DEBUG "${CMAKE_CXX_FLAGS_DEBUG} -Wall
        -Wextra -g")
    set(CMAKE_CXX_FLAGS_RELEASE "${CMAKE_CXX_FLAGS_RELEASE}
        -O2")
endif()

# Add sources
file(GLOB SOURCES "${PROJECT_SOURCE_DIR}/src/*.cpp" "${PROJECT_S
   OURCE_DIR}/*.cpp")

# Specify include directory
include_directories("${PROJECT_SOURCE_DIR}/include/")

# Compile and generate the executable
add_executable(kpeg main.cpp src/Decoder.cpp src/Image.cpp src/H
    uffmanTree.cpp src/MCU.cpp src/Transform.cpp src/Utility.cpp)

# Specify that a C++14 compliant compiler is required
set_property(TARGET kpeg PROPERTY CXX_STANDARD 14)
set_property(TARGET kpeg PROPERTY CXX_STANDARD_REQUIRED ON)

```

What we are doing in the script is specifying our project name, the compilation flags to use when building in debug & release modes, the location of the sources, the location of the headers, and finally the binary executable that will be generated. We are also telling CMake that we want to use only compilers that are C++14 or later compliant.

The detailed explanation for CMake syntax and structure used above is out of the scope of this article, so I would suggest you to take a look at the [official CMake tutorials](#) if you're not very familiar with it.

NOTE

You are free to skip setting up a build system using CMake as it might be an unnecessary hindrance for the actual focus (writing the decoder). Having said that, using CMake ensures that your code can be built on any platform, using any C++ compiler vendor (provided that they support new features that

are used from C++14). But if you do so, you will need to set up the build process accordingly for the environment that you are developing on (it can be as simple as manually invoking the required command to build your code).

Adding Common Structures and Types

Now that the process for building the application is set up, it's time to move on to actual code!

Previously, we created some empty C++ sources & headers. Let's take a look at the classes & structures we will be creating inside them, and what piece of functionality they provide.

Utility

The `Utility` module contains various utility functions for characters and strings. It has the following utility functions:

- `isValidChar` : checks the validity of a character in the context of a file name
- `isValidFilename` : checks whether a string represents a valid JPEG file name
- `isWhiteSpace` : checks whether a character represents a whitespace
- `isStringWhiteSpace` : checks whether a string consists entirely of whitespaces

`Utility.hpp`

```
#ifndef UTILITY_HPP
#define UTILITY_HPP

#include <string>
#include <cctype>

// Output log file, ugly solution global for logging
// into files! This approach is not recommended, I
// used here to focus on JPEG, not logging! :p
extern std::ofstream logFile;

namespace kpeg
{

inline const bool isValidChar(const char ch)
{
    return isprint(ch) || ch != '/' || ch != '\\';
}

inline const bool isValidFilename(const std::string& filename)
{
    for (auto&& c : filename)
        if (!isValidChar(c))
            return false;
}
```

```

std::size_t extPos = filename.find(".jpg");

if (extPos == std::string::npos)
    extPos = filename.find(".jpeg");

else if (extPos + 4 == filename.size())
    return true;

if (extPos == std::string::npos)
    return false;

return false;
}

inline const bool isWhiteSpace(const char ch)
{
    return iscntrl(ch) || isblank(ch) || isspace(ch);
}

inline const bool isStringWhiteSpace(const std::string& str)
{
    for (auto& c : str)
        if (!isWhiteSpace(c))
            return false;
    return true;
}

}

#endif // UTILITY_HPP

```

Utility.cpp

```

#include "Utility.hpp"

std::ofstream logFile("kpeg.log", std::ios::out);

```

Transform

This module contains various functions for transforming pieces of compressed image data between different forms, and their helper functions.

First, we will define two helper functions to convert between matrix indices and their corresponding zig-zag order:

- `matIndicesToZZOrder` : converts a matrix index, (i, j) , to its corresponding order, z , in zig-zag ordering
- `zzOrderToMatIndices` : converts a zig-zag order, z , to its corresponding matrix index, (i, j)

Then, we have two important functions that will be used in the decoder:

- `bitStringToValue` : convert a bit string representation to its corresponding value
- `getValueCategory` : get the category of a value

If you recall from part one, we need to know the category of a value and its bit string representation to be able to determine the Huffman code for the value. Hence, we will need to be able to the exact reverse: determine the value from the bitstring (once the value has been determined using the Huffman tables supplied in the JPEG file).

Transform.hpp

```
#ifndef TRANSFORM_HPP
#define TRANSFORM_HPP

#include <string>
#include <utility>

#include "Types.hpp"

namespace kpeg
{

    const std::pair<const int, const int> zzOrderToMatIndices(const
    int zzindex);

    const int matIndicesToZZOrder(const int row, const int column);

    const Int16 bitStringtoValue(const std::string& bitStr);

    const Int16 getValueCategory(const Int16 value);

}

#endif // TRANSFORM_HPP
```

Transform.cpp

```
#include <cmath>

#include "Transform.hpp"

namespace kpeg
{

    const std::pair<const int, const int> zzOrderToMatIndices(const
    int zzindex)
    {
        static const std::pair<const int, const int> zzorder[64] =
        {
            {0,0},
            {0,1}, {1,0},
            {2,0}, {1,1}, {0,2},
            {0,3}, {1,2}, {2,1}, {3,0},
            {4,0}, {3,1}, {2,2}, {1,3}, {0,4},
            {0,5}, {1,4}, {2,3}, {3,2}, {4,1}, {5,0},
            {6,0}, {5,1}, {4,2}, {3,3}, {2,4}, {1,5}, {0,6},
            {0,7}, {1,6}, {2,5}, {3,4}, {4,3}, {5,2}, {6,1}, {7,0},
            {7,1}, {6,2}, {5,3}, {4,4}, {3,5}, {2,6}, {1,7},
            {2,7}, {3,6}, {4,5}, {5,4}, {6,3}, {7,2},
            {7,3}, {6,4}, {5,5}, {4,6}, {3,7},
            {4,7}, {5,6}, {6,5}, {7,4},
            {7,5}, {6,6}, {5,7},
```

```

        {6,7}, {7,6},
        {7,7}
    };

    return zzorder[zzindex];
}

const int matIndicesToZZOrder(const int row, const int column)
{
    static const int matOrder[8][8] =
    {
        { 0, 1, 5, 6, 14, 15, 27, 28 },
        { 2, 4, 7, 13, 16, 26, 29, 42 },
        { 3, 8, 12, 17, 25, 30, 41, 43 },
        { 9, 11, 18, 24, 31, 40, 44, 53 },
        { 10, 19, 23, 32, 39, 45, 52, 54 },
        { 20, 22, 33, 38, 46, 51, 55, 60 },
        { 21, 34, 37, 47, 50, 56, 59, 61 },
        { 35, 36, 48, 49, 57, 58, 62, 63 }
    };

    return matOrder[row][column];
}

const Int16 bitStringtoValue(const std::string& bitStr)
{
    if (bitStr == "")
        return 0x0000;

    Int16 value = 0x0000;

    char sign = bitStr[0];
    int factor = sign == '0' ? -1 : 1;

    for (auto i = 0; i < bitStr.size(); ++i)
    {
        if (bitStr[i] == sign)
            value += Int16(std::pow(2, bitStr.size() - 1 - i));
    }

    return factor * value;
}

const Int16 getValueCategory(const Int16 value)
{
    if (value == 0x0000)
        return 0;
    return std::log2(std::abs(value)) + 1;
}
}

```

Types

The `Types` module contains definitions and aliases for commonly used types in the scope of our decoder.

It provides the following type aliases and types:

- `UInt8` , `UInt16` : unsigned integral types 8-bits and 16-bits wide
- `RGBComponents` : identifying the components of RGB colour model pixels

- `Pixel` : represents a three-channel pixel with discrete integral range of channel intensities
- `PixelPtr` : represents a 2D array of `Pixel` s
- `HuffmanTable` : represents a Huffman table with upto 16 length symbols.

`Types.hpp`

```

/// Types module
///
/// Contains definition of standard types

#ifndef TYPES_HPP
#define TYPES_HPP

#include <vector>
#include <array>
#include <utility>
#include <memory>

namespace kpeg
{
    namespace types
    {
        typedef unsigned char  UInt8;
        typedef unsigned short UInt16;

        typedef char  Int8;
        typedef short Int16;

        enum RGBComponents
        {
            RED    ,
            GREEN  ,
            BLUE
        };

        struct Pixel
        {
            Pixel()
            {
                comp[0] = comp[1] = comp[2] = 0;
            }

            Pixel(const Int16 comp1,
                 const Int16 comp2,
                 const Int16 comp3)
            {
                comp[0] = comp1;
                comp[1] = comp2;
                comp[2] = comp3;
            }

            // Store the intensity of the pixel
            Int16 comp[3];
        };

        typedef std::shared_ptr<std::vector<std::vector<Pixel>>> PixelPtr;
        typedef std::array<std::pair<int, std::vector<UInt8>>, 16> HuffmanTable;
    }
}

```

```
// Identifiers used to access a Huffman table based on
// the class and ID. E.g., To access the Huffman table
// for the DC coefficients of the CbCr component, we
// use `huff_table[HT_DC][HT_CbCr]`.
const int HT_DC    = 0;
const int HT_AC    = 1;
const int HT_Y     = 0;
const int HT_CbCr  = 1;

}

}

#endif // TYPES_HPP
```

Markers

The JFIF file marker constants are defined in the `Markers` module. Irrelevant markers (e.g., `RST` markers) have been omitted out.

`Markers.hpp`

```
// The JFIF file markers

#ifndef MARKERS_HPP
#define MARKERS_HPP

#include <string>

#include "Image.hpp"

namespace kpeg
{

// Null byte
const UInt16 JFIF_BYTE_0    = 0x00;

// JPEG-JFIF File Markers
//
// Refer to ITU-T.81 (09/92), page 32
const UInt16 JFIF_BYTE_FF    = 0xFF; // All markers start with this as the MSB
const UInt16 JFIF_SOF0       = 0xC0; // Start of Frame 0, Baseline DCT
const UInt16 JFIF_SOF1       = 0xC1; // Start of Frame 1, Extended Sequential DCT
const UInt16 JFIF_SOF2       = 0xC2; // Start of Frame 2, Progressive DCT
const UInt16 JFIF_SOF3       = 0xC3; // Start of Frame 3, Lossless (Sequential)
const UInt16 JFIF_DHT         = 0xC4; // Define Huffman Table
const UInt16 JFIF_SOF5       = 0xC5; // Start of Frame 5, Differential Sequential DCT
const UInt16 JFIF_SOF6       = 0xC6; // Start of Frame 6, Differential Progressive DCT
const UInt16 JFIF_SOF7       = 0xC7; // Start of Frame 7, Differential Lossless (Sequential)
const UInt16 JFIF_SOF9       = 0xC9; // Extended Sequential DCT, Arithmetic Coding
const UInt16 JFIF_SOF10      = 0xCA; // Progressive DCT, Arithmetic Coding
const UInt16 JFIF_SOF11      = 0xCB; // Lossless (Sequential), A
```



```

rithmetic Coding
const UInt16 JFIF_SOF13      = 0xCD; // Differential Sequential
DCT, Arithmetic Coding
const UInt16 JFIF_SOF14      = 0xCE; // Differential Progressive
DCT, Arithmetic Coding
const UInt16 JFIF_SOF15      = 0xCF; // Differential Lossless (S
quential), Arithmetic Coding
const UInt16 JFIF_SOI        = 0xD8; // Start of Image
const UInt16 JFIF_EOI        = 0xD9; // End of Image
const UInt16 JFIF_SOS        = 0xDA; // Start of Scan
const UInt16 JFIF_DQT        = 0xDB; // Define Quantization Tabl
e
const UInt16 JFIF_APP0       = 0xE0; // Application Segment 0, J
PEG-JFIF Image
const UInt16 JFIF_COM        = 0xFE; // Comment

}

#endif // MARKERS_HPP

```

Huffman Tree

Let's go over how Huffman tables are used in JPEG. If you remember from my last post, Huffman coding is applied to assign variable length codes to a group symbols in such a way that the more frequently used symbols are assigned shorter codes than the less frequent ones. This is what's used by JPEG too - Huffman encode the run-length encoded data. Now, the Huffman table of codes and their corresponding symbols aren't stored directly in a JFIF file. If you recollect the `DHT` segment of JFIF, you will notice that a byte describing Huffman table (henceforth referred to as HT) info (HT is for DC or AC), followed by 16 bytes where each byte represents the total number of symbols for its index, i.e., first byte represents total number of symbols of length 1, second one represents total number of symbols of length 2, and so on. Let's assume that the total number of symbols of all lengths, 1 through 16, is n . Then, the next n bytes that follow, they describe the symbols, in sequence, for the Huffman table.

Let's take a minimal example. Suppose we have a Huffman table as shown:

Symbol	Huffman code	Code length
a	00	2
b	010	3
c	011	3
d	100	3
e	101	3
f	110	3
g	1110	4

Symbol	Huffman code	Code length
h	11110	5
i	111110	6
j	1111110	7
k	11111110	8
l	111111110	9

Is table will get stored in the JFIF file like so (of course they won't be stored as plain ASCII values but in their binary form):

```
0 1 5 1 1 1 1 1 0 0 0 0 0 0 0 a b c d e f g h i j k
l
```

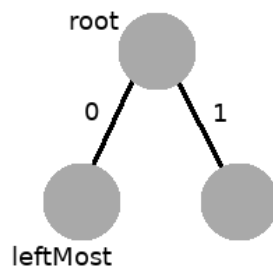
Now, with only above the table data, it is the decoder's job to reconstruct the Huffman tree and hence obtain the Huffman codes assigned to the symbols. Let's see how to construct a Huffman tree knowing only the symbol count for each code length and the symbols themselves.

```
1. Create empty root node, root
2. Add empty left & right child of root
3. SET leftMost := root.left
3. FOR i in RANGE(1, 16) DO
4.     IF SymbolCount(i) EQUALS 0 THEN // SymbolCount(n) return
s # of codes with length n
5.         current := leftMost
6.         WHILE current NOT EQUALS NULL DO
7.             Add empty left & right child of current
8.             current := RightLevelNode(current) // RightLevel
Node(n) returns the right node of n in the current level, or nul
l if none exist
9.         END-WHILE
10.        leftMost := leftMost.left
11.    ELSE DO
12.        FOR symbol in GetSymbols(n) DO // GetSymbols(n) retu
rns the list of symbols with length n
13.            leftMost.value := symbol
14.            leftMost := RightLevelNode(leftMost)
15.        END-FOR
16.        Add empty left & right child of leftMost
17.        current := leftMost.right
18.        leftMost := leftMost.left
19.        WHILE current NOT EQUALS NULL DO
20.            add empty left & right child of current
21.        END-WHILE
22.    END-IF
23. END-FOR
```

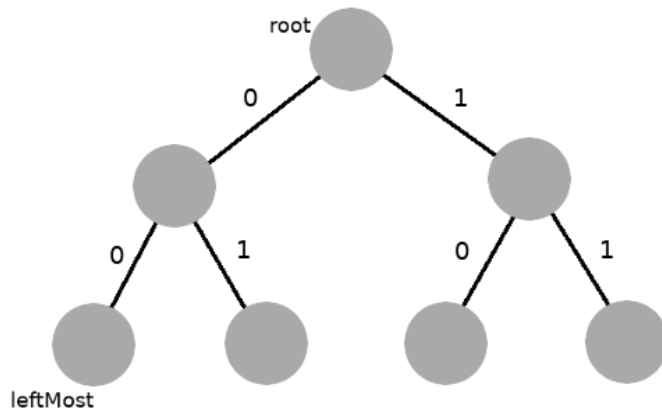
The logic used is pretty simple. Huffman trees are just binary trees. You start at the root of the tree, and start assigning symbols from the leftmost node, moving within the same depth towards the rightmost node. While doing so, if the current node is not the right most node, we simply add children to all the remaining nodes in the same level. Then we start the process

again from the current node. This is better illustrated with the following graphical representation.

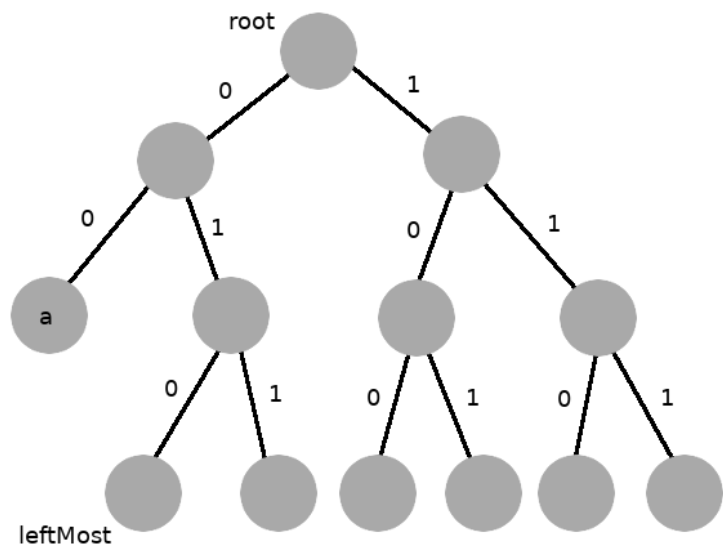
Start with a root node with empty left & right children.



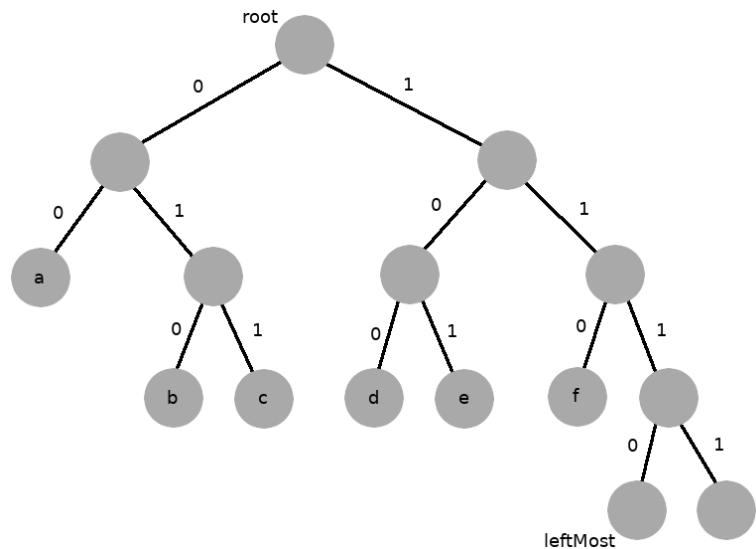
Since, symbol count for code length 1 is 0, we execute steps 5 to 10.



Now, for code length 2, we have one symbol, `a`, hence we execute steps 12 to 21.



For code length 3, we have five symbols, `b`, `c`, `d`, `e`, and `f`, and again we execute steps 12 to 21.



Similarly, the entire Huffman tree with all the symbols can be constructed.

If you look closely, you will notice that traversing from the root node to one of the leaf nodes that contain a symbol gives us the Huffman code!

So, if the decoder reads the next three bytes in the JFIF scan data and encounters the bitstream `101`, then it simply traverses the Huffman tree and reaches a leaf node with value `e`, meaning that the code `101` corresponds to symbol `e`. Also, note the fact that all symbols appear only as leaf nodes. This is due to the fact that no symbols are prefixes to other symbols in Huffman coding, i.e., we can't have two codes like `0111` and `01111`.

With the explanation of Huffman trees and tables, let's take a look at how we will implement it in code.

Node

Firstly, we will use a helper structure called `Node` that will have the relevant properties and operations defined:

Properties

- `root` : boolean value to indicate whether the node is a root node
- `leaf` : boolean value to indicate whether the node is a leaf node
- `code` : the Huffman code corresponding to the node
- `value` : the symbol value of a leaf node in the Huffman tree
- `lChild` : left child of the node
- `rChild` : right child of the node
- `parent` : parent of the node

Operations

- `createRootNode` : create an new binary tree
- `createNode` : helper function to create a node
- `insertLeft` : add a left child for the specified node
- `insertRight` : add a right child for the specified node
- `getRightLevelNode` : get the node at the immediate right & at the same level to the specified node
- `inOrder` : perform inorder traversal of the Huffman tree with specified root node

The implementation for the `Node` structure is pretty straightforward:

HuffmanTree.hpp

```
#ifndef HUFFMAN_TREE_HPP
#define HUFFMAN_TREE_HPP

#include <string>
#include <memory>

#include "Types.hpp"

namespace kpeg
{
    struct Node
    {
        Node() :
            root{ false },
            leaf{ false },
            code{ "" },
            value{ 0x00 },
            lChild{ nullptr },
            rChild{ nullptr },
            parent{ nullptr }
        {}

        Node(const std::string _code, const UInt16 _val) :
            root{ false },
            leaf{ false },
            code{ _code },
            value{ _val },
            lChild{ nullptr },
            rChild{ nullptr },
            parent{ nullptr }
        {}

        bool root;
        bool leaf;
        std::string code;
        UInt16 value;
        std::shared_ptr<Node> lChild, rChild;
        std::shared_ptr<Node> parent;
    };

    // Alias for a node
    typedef std::shared_ptr<Node> NodePtr;

    inline NodePtr createRootNode(const UInt16 value)
    {

```

```

        NodePtr root = std::make_shared<Node>( "", value );
        root->root = true;
        return root;
    }

    inline NodePtr createNode()
    {
        return std::make_shared<Node>();
    }

    void insertLeft(NodePtr node, const UInt16 value);

    void insertRight(NodePtr node, const UInt16 value);

    NodePtr getRightLevelNode(NodePtr node);

    void inOrder(NodePtr node);

}

```

HuffmanTree.cpp

```

#include <iomanip>

#include "HuffmanTree.hpp"
#include "Utility.hpp"

namespace kpeg
{
    void insertLeft( NodePtr node, const UInt16 value )
    {
        if ( node == nullptr )
            return;

        if ( node->lChild != nullptr )
        {
            logFile << "Given node already has a left child, skipping insertion" << std::endl;
            return;
        }

        NodePtr lNode = createNode();
        lNode->parent = node;
        node->lChild = lNode;

        lNode->code = node->code + "0";
        lNode->value = value;
    }

    void insertRight( NodePtr node, const UInt16 value )
    {
        if ( node == nullptr )
            return;

        if ( node->rChild != nullptr )
        {
            logFile << "Given node already has a right child, skipping insertion" << std::endl;
            return;
        }

        NodePtr rNode = createNode();
        rNode->parent = node;
        node->rChild = rNode;
    }
}

```

```

        rNode->code = node->code + "1";
        rNode->value = value;
    }

    NodePtr getRightLevelNode( NodePtr node )
    {
        if ( node == nullptr )
            return nullptr;

        // Node is the left child of its parent, then the parent's
        // right child is its right level order node.
        if ( node->parent != nullptr && node->parent->lChild == node
        )
            return node->parent->rChild;

        // Else node is the right child of its parent, then traverse
        // back the tree and find its right level order node
        int count = 0;
        NodePtr nptr = node;
        while ( nptr->parent != nullptr && nptr->parent->rChild == n
ptr )
        {
            nptr = nptr->parent;
            count++;
        }

        if ( nptr->parent == nullptr )
            return nullptr;

        nptr = nptr->parent->rChild;

        int i = 1;
        while ( count > 0 )
        {
            nptr = nptr->lChild;
            count--;
        }

        return nptr;
    }

    void inOrder( NodePtr node )
    {
        if ( node == nullptr )
            return;
        inOrder(node->lChild);

        if ( node->code != "" && node->leaf )
            logFile << "Symbol: 0x" << std::hex << std::setfill('0')
<< std::setw(2) << std::setprecision(16) << node->value << ", Co
de: " << node->code << std::endl;

        inOrder(node->rChild);
    }
}

```

HuffmanTree

Now let's take a look at the class `HuffmanTree` :

Properties

- `m_root` : the root node of the Huffman tree

Operations

- `constructHuffmanTree` : creates a Huffman tree out of a Huffman table (as demonstrated in the above example)
- `getTree` : get the root node of the Huffman tree
- `contains` : checks whether a given Huffman code is present in the tree or not

HuffmanTree.hpp

```
namespace kpeg
{

class HuffmanTree
{
public:

    HuffmanTree();

    HuffmanTree(const HuffmanTable& htable);

    void constructHuffmanTree(const HuffmanTable& htable);

    const NodePtr getTree() const;

    // NOTE: std::string is used as the return type because
    // 0x0000 and 0xFFFF
    // are both values that are used in the normal range. So
    // using them is not
    // possible to indicate special conditions (e.g., code not
    // found in tree)
    const std::string contains(const std::string& huffCode);

private:

    NodePtr m_root;
};

}
```

HuffmanTree.cpp

```
namespace kpeg
{

HuffmanTree::HuffmanTree() :
    m_root{nullptr}
{
}

HuffmanTree::HuffmanTree( const HuffmanTable& htable )
{
    constructHuffmanTree( htable );
}

void HuffmanTree::constructHuffmanTree( const HuffmanTable& htable )
{
    logFile << "Constructing Huffman tree with specified Huffman
table..." << std::endl;

    m_root = createRootNode( 0x0000 );
}
```



```

        insertLeft( m_root, 0x0000 );
        insertRight( m_root, 0x0000 );
        inOrder( m_root );
        NodePtr leftMost = m_root->lChild;

        for ( auto i = 1; i <= 16; ++i )
        {
            // If the count is zero, add left & right children for a
            // 11 unassigned leaf nodes.
            if ( htable[i - 1].first == 0 )
            {
                for ( NodePtr nptr = leftMost; nptr != nullptr; nptr
= getRightLevelNode( nptr ) )
                {
                    insertLeft( nptr, 0x0000 );
                    insertRight( nptr, 0x0000 );
                }

                leftMost = leftMost->lChild;
            }

            // Else assign codes to nodes starting from leftmost lea
            f in the tree.
            else
            {
                for ( auto&& huffVal : htable[i - 1].second )
                {
                    leftMost->value = huffVal;
                    leftMost->leaf = true;
                    leftMost = getRightLevelNode( leftMost );
                }

                insertLeft( leftMost, 0x0000 );
                insertRight( leftMost, 0x0000 );

                NodePtr nptr = getRightLevelNode( leftMost );
                leftMost = leftMost->lChild;

                while ( nptr != nullptr )
                {
                    insertLeft( nptr, 0x0000 );
                    insertRight( nptr, 0x0000 );

                    nptr = getRightLevelNode( nptr );
                }
            }
        }

        logFile << "Finished building Huffman tree [OK]" << std::end
        l;
    }

    const NodePtr HuffmanTree::getTree() const
    {
        return m_root;
    }

    const std::string HuffmanTree::contains( const std::string& huff
    Code )
    {
        if ( utils::isStringWhiteSpace( huffCode ) )
        {
            logFile << "[ FATAL ] Invalid huffman code, possibly cor
            rupt JFIF data stream!" << std::endl;
            return "";
        }
    }

```

```

int i = 0;
NodePtr nptr = m_root;

do
{
    if ( huffCode[i] == '0' )
        nptr = nptr->lChild;
    else
        nptr = nptr->rChild;

    if ( nptr != nullptr && nptr->leaf && nptr->code == huff
Code )
    {
        if ( nptr->value == 0x0000 )
            return "EOB";
        return std::to_string( nptr->value );
    }
    i++;
} while ( nptr != nullptr && i < huffCode.size() );

return "";
}
}

```

MCU

As discussed previously, MCU is the unit of operations in JPEG. We will define a class to capture its properties and behaviours.

The properties and behaviours that are required are as follows:

Properties

- `m_block` : 8×8 block of pixels that make up the MCU
- `m_order` : the order of the MCU in the image
- `m_IDCTCoeffs` : the MCU after performing IDCT
- `m_DCDiff` : the differences in DC consecutive coefficients per channel (this info is shared by all MCUs)
- `m_QTables` : the quantization tables used for quantization (this info is shared by all MCUs, and publicly available)
- `m_MCUCount` : the total number of MCUs in the image (this info is shared by all MCUs, an publicly available)

Operations

- `constructMCU` : create the MCU from the specified run-length encoding and quantization tables
- `getAllMatrices` : get the pixel arrays for the pixels under this MCU
- `computeIDCT` : compute IDCT
- `performLevelShift` : level shift the pixel data to center it within the pixel value range

- `convertYCbCrToRGB` : convert the MCU's underlying pixels from the Y-Cb-Cr colour model to RGB colour model

Now let's write the class declaration.

MCU.hpp

```
#ifndef MCU_HPP
#define MCU_HPP

#include <array>
#include <vector>
#include <utility>

#include "Types.hpp"
#include "Transform.hpp"

namespace kpeg
{

// Alias for a 8x8 pixel block with integral values for its channels
typedef std::array<std::array<std::array<int, 8>, 8>, 3> CompMatrices;

// Alias for a 8x8 matrix with integral elements
typedef std::array< std::array< int, 8 >, 8 > Matrix8x8;

class MCU
{
public:

    static int m_MCUCount;
    static std::vector<std::vector<UInt16>> m_QTables;

public:

    MCU();

    MCU(const std::array<std::vector<int>, 3>& compRLE,
        const std::vector<std::vector<UInt16>>& QTables);

    void constructMCU(const std::array<std::vector<int>, 3>& compRLE,
        const std::vector<std::vector<UInt16>>& QTables);

    const CompMatrices& getAllMatrices() const;

private:

    void computeIDCT();

    void performLevelShift();

    void convertYCbCrToRGB();

private:

    CompMatrices m_block;
    int m_order;
    static int m_DCDiff[3];
    std::array<std::array<std::array<float, 8>, 8>, 3> m_IDCTCoeffs;
};
```

```

    }

#endif // MCU_HPP

```

And the implementation:

MCU.cpp

```

#include <string>
#include <sstream>
#include <cmath>
#include <iomanip>
#include <iostream>

#include "Utility.hpp"
#include "MCU.hpp"

namespace kpeg
{
    int MCU::m_MCUCount = 0;
    std::vector<std::vector<UInt16>> MCU::m_QTables = {};
    int MCU::m_DCDiff[3] = { 0, 0, 0 };

    MCU::MCU()
    {
    }

    MCU::MCU(const std::array<std::vector<int>, 3>& compRLE, const std::vector<std::vector<UInt16>>& QTables)
    {
        constructMCU(compRLE, QTables);
    }

    void MCU::constructMCU(const std::array<std::vector<int>, 3>& compRLE, const std::vector<std::vector<UInt16>>& QTables)
    {
        m_QTables = QTables;

        m_MCUCount++;
        m_order = m_MCUCount;

        logFile << "Constructing MCU: " << std::dec << m_order <<
            "... " << std::endl;

        const char* component[] = { "Y (Luminance)", "Cb (Chrominance)", "Cr (Chrominance)" };
        const char* type[] = { "DC", "AC" };

        for (int compID = 0; compID < 3; compID++)
        {
            // Initialize with all zeros
            std::array<int, 64> zzOrder;
            std::fill(zzOrder.begin(), zzOrder.end(), 0);
            int j = -1;

            for (auto i = 0; i <= compRLE[compID].size() - 2; i += 2)
            {
                if (compRLE[compID][i] == 0 && compRLE[compID][i + 1] == 0)
                    break;

                j += compRLE[compID][i] + 1; // Skip the number of p

```

```

ositions containing zeros
        zzOrder[j] = compRLE[compID][i + 1];
    }

    // DC_i = DC_i-1 + DC-difference
    m_DCDiff[compID] += zzOrder[0];
    zzOrder[0] = m_DCDiff[compID];

    int QIndex = compID == 0 ? 0 : 1;
    for (auto i = 0; i < 64; ++i) // !!!!! i = 1
        zzOrder[i] *= m_QTables[QIndex][i];

    // Zig-zag order to 2D matrix order
    for (auto i = 0; i < 64; ++i)
    {
        auto coords = zzOrderToMatIndices(i);

        m_block[compID][ coords.first ][ coords.second ] = z
zzOrder[i];
    }
}

computeIDCT();
performLevelShift();
convertYCbCrToRGB();

logFile << "Finished constructing MCU: " << m_order << "...";
<< std::endl;
}

const CompMatrices& MCU::getAllMatrices() const
{
    return m_block;
}

void MCU::computeIDCT()
{
    logFile << "Performing IDCT on MCU: " << m_order << "...";
    std::endl;

    for (int i = 0; i < 3; ++i)
    {
        for (int y = 0; y < 8; ++y)
        {
            for (int x = 0; x < 8; ++x)
            {
                float sum = 0.0;

                for (int u = 0; u < 8; ++u)
                {
                    for (int v = 0; v < 8; ++v)
                    {
                        float Cu = u == 0 ? 1.0 / std::sqrt(2.0)
: 1.0;
                        float Cv = v == 0 ? 1.0 / std::sqrt(2.0)
: 1.0;

                        sum += Cu * Cv * m_block[i][u][v] * st
d::cos((2 * x + 1) * u * M_PI / 16.0) *
std::cos((2 * y + 1) * v
* M_PI / 16.0);
                    }
                }

                m_IDCTCoeffs[i][x][y] = 0.25 * sum;
            }
        }
    }
}

```

```

    }

    logFile << "IDCT of MCU: " << m_order << " complete [OK]" <<
std::endl;
}

void MCU::performLevelShift()
{
    logFile << "Performing level shift on MCU: " << m_order <<
"... " << std::endl;

    for (int i = 0; i < 3; ++i)
    {
        for (int y = 0; y < 8; ++y)
        {
            for (int x = 0; x < 8; ++x)
            {
                m_block[i][y][x] = std::roundl(m_IDCTCoeffs[i]
[y][x]) + 128;
            }
        }
    }

    logFile << "Level shift on MCU: " << m_order << " complete
[OK]" << std::endl;
}

void MCU::convertYCbCrToRGB()
{
    logFile << "Converting from Y-Cb-Cr colorspace to R-G-B colo
rspace for MCU: " << m_order << "... " << std::endl;

    for (int y = 0; y < 8; ++y)
    {
        for (int x = 0; x < 8; ++x)
        {
            float Y = m_block[0][y][x];
            float Cb = m_block[1][y][x];
            float Cr = m_block[2][y][x];

            int R = (int)std::floor(Y + 1.402 * (1.0 * Cr - 128.
0));
            int G = (int)std::floor(Y - 0.344136 * (1.0 * Cb - 1
28.0) - 0.714136 * (1.0 * Cr - 128.0));
            int B = (int)std::floor(Y + 1.772 * (1.0 * Cb - 128.
0));

            R = std::max(0, std::min(R, 255));
            G = std::max(0, std::min(G, 255));
            B = std::max(0, std::min(B, 255));

            m_block[0][y][x] = R;
            m_block[1][y][x] = G;
            m_block[2][y][x] = B;
        }
    }

    logFile << "Colorspace conversion for MCU: " << m_order << "
done [OK]" << std::endl;
}
}

```

Image

The class `Image` is an abstraction for dealing with a raw image data, and provides high level functions for manipulating a raw image. `Image` is designed to be able to hold the compressed data in a JFIF file and write. In general, `Image` is designed to hold raw, uncompressed data for any image that uses a three-channel colour model.

The properties of `Image` are as follows:

- `m_width` : the width of the image in pixels
- `m_height` : the height of the image in pixels
- `m_pixelPtr` : the 2D array of pixels of the image where the pixel intensities are discrete

The operations defined on `Image` :

- `createImageFromMCUs` : create an image from a list of MCUs
- `dumpRawData` : Write the raw, uncompressed image data to specified file on the disk. The data written is in PPM format

Now, for the declaration & implementation of the `Image` class:

`Image.hpp`

```
#ifndef IMAGE_HPP
#define IMAGE_HPP

#include <iostream>
#include <fstream>
#include <vector>
#include <array>
#include <memory>

#include "Types.hpp"
#include "MCU.hpp"

namespace kpeg
{
    class Image
    {
    public:
        Image();

        void createImageFromMCUs(const std::vector<MCU>& MCUs);

        const bool dumpRawData(const std::string& filename);

    public:
        std::size_t width;
        std::size_t height;

    private:
        PixelPtr m_pixelPtr;
    };
}
```

```
#endif // IMAGE_HPP
```

Image.cpp

```
#include <arpa/inet.h> // htons
#include <string>
#include <cmath>

#include "Utility.hpp"
#include "Image.hpp"

namespace kpeg
{
    Image::Image() :
        width{0},
        height{0},
        m_pixelPtr{nullptr}
    {
        logFile << "Created new Image object" << std::endl;
    }

    void Image::createImageFromMCUs(const std::vector<MCU>& MCUs)
    {
        logFile << "Creating Image from MCU vector..." << std::endl;

        int mcuNum = 0;

        int jpegWidth = width % 8 == 0 ? width : width + 8 - (width
% 8);
        int jpegHeight = height % 8 == 0 ? height : height + 8 - (height
% 8);

        // Create a pixel pointer of size (Image width) x (Image height)
        m_pixelPtr = std::make_shared<std::vector<std::vector<Pixel>
>>(
            jpegHeight, std::vector<Pixel>(jpegWidth, Pixel()));

        // Populate the pixel pointer based on data from the specified MCUs
        for (int y = 0; y <= jpegHeight - 8; y += 8)
        {
            for (int x = 0; x <= jpegWidth - 8; x += 8)
            {
                auto pixelBlock = MCUs[mcuNum].getAllMatrices();

                for (int v = 0; v < 8; ++v)
                {
                    for (int u = 0; u < 8; ++u)
                    {
                        (*m_pixelPtr)[y + v][x + u].comp[0] = pixelBlock[0][v][u]; // R
                        (*m_pixelPtr)[y + v][x + u].comp[1] = pixelBlock[1][v][u]; // G
                        (*m_pixelPtr)[y + v][x + u].comp[2] = pixelBlock[2][v][u]; // B
                    }
                }

                mcuNum++;
            }
        }

        // Trim the image width to nearest multiple of 8
    }
}
```



```

        if (width != jpegWidth)
        {
            for (auto&& row : *m_pixelPtr)
                for (int c = 0; c < 8 - width % 8; ++c)
                    row.pop_back();
        }

        // Trim the image height to nearest multiple of 8
        if (height != jpegHeight)
        {
            for (int c = 0; c < 8 - height % 8; ++c)
                m_pixelPtr->pop_back();
        }

        logFile << "Finished created Image from MCU [OK]" << std::endl;
    }

    const bool Image::dumpRawData(const std::string& filename)
    {
        if (m_pixelPtr == nullptr)
        {
            logFile << "Unable to create dump file '\" + filename +
            '\", Invalid pixel pointer" << std::endl;
            return false;
        }

        std::ofstream dumpFile(filename, std::ios::out);

        if (!dumpFile.is_open() || !dumpFile.good())
        {
            logFile << "Unable to create dump file '\" + filename +
            '\", Invalid file pointer" << std::endl;
            return false;
        }

        dumpFile << "P6" << std::endl;
        dumpFile << "# PPM dump created using libKPEG: https://github.com/TheIllusionistMirage/libKPEG" << std::endl;
        dumpFile << width << " " << height << std::endl;
        dumpFile << 255 << std::endl;

        for (auto&& row : *m_pixelPtr)
        {
            for (auto&& pixel : row)
                dumpFile << (UInt8)pixel.comp[RGBComponents::RED]
                << (UInt8)pixel.comp[RGBComponents::GREEN]
                << (UInt8)pixel.comp[RGBComponents::BLUE]
                << " ";
            dumpFile << "\n";
        }

        logFile << "Raw image data dumped to file: '\" + filename +
        '\", Invalid file pointer" << std::endl;
        dumpFile.close();
        return true;
    }
}

```

The implementation should be straightforward. If any questions arise regarding the implementation of the

`createImageFromMCUs` method, here's an explanation: MCUs are 8×8 blocks of pixels, and one block exists per component in the colour model. And the abstraction that we have defined (`MCU`) has three 8×8 blocks of pixels, one for

each in the RGB colour model. The image has a blank pixel pointer that spans the entire width and height of the image. We simply loop through the MCUs, copying each pixel data in MCUs to their proper place in the pixel pointer array.

Another point to notice is the `dumpRawData` method. It writes to disk the uncompressed image data in the [PPM image format](#). The only reason why I didn't add a dedicated section to explain it is very minimal.

Decoder

At this point let's pause for a moment and take a look at what we have so far:

- We defined types and type aliases for dealing with raw binary data
- We defined methods for transforming pieces of image between different forms
- We defined abstractions for dealing with MCUs and uncompressed image data

The logic for decoding an image is something like this:

```

1.  Open JFIF file
2.  WHILE !EOF AND no-errors DO
3.      READ next byte
4.      IF byte EQUALS `ff` THEN
5.          READ next byte
6.          IF byte EQUALS a defined marker THEN
7.              parse marker data
8.          END-IF
9.      END-IF
10. END-WHILE
11. Decode scan data
12. Construct `MCU`s from decoded data
13. Construct `Image` from the MCUs
14. Write back raw, uncompressed data to disk (in PPM format)

```

The Interface for the Decoder

I roughly described the logic for decoding a JFIF file in the last section. Let's actually lay down the interface of the decoder for a better understanding.

The decoder should be able to:

- Detect markers and parse them
- Read and decode scan data (the JPEG encoded image data)
- Determine the Huffman tables and quantization tables to use for decoding

- Create MCUs from the scan data using the Huffman tables & quantization tables obtained
- Write the decoded image in raw, uncompressed form to disk in the PPM format

For doing all the above, we will add the following methods to our `Decoder` class:

- `open` : open a JFIF file for decoding
- `decodeImageFile` : decode the image in the JFIF file
- `dumpRawData` : write raw, uncompressed image data to disk in PPM format
- `close` : close the JFIF file after decoding

Besides this public facing interface, we will have private methods that will do the heavy lifting:

- `parseSegmentInfo` : parse the info of the specified segment in the JFIF file
- `parseAPP0Segment` : parse the JFIF segment at the very beginning of the JFIF file
- `parseCOMSegment` : parse the comment in the JFIF file
- `parseDQTSegment` : parse the quantization tables specified in the JFIF file
- `parseSOF0Segment` : parse the Start of File segment
- `parseDHTSegment` : parse the Huffman tables specified in the JFIF file
- `parseSOSSegment` : parse the start of scan segment in the JFIF file
- `scanImageData` : parse the actual compressed image data stored in the JFIF file
- `byteStuffScanData` : convert bytes of the form `XXFF00YY` to just `XXFFYY`
- `decodeScanData` : decode the RLE-Huffman encoded image pixel data

So, the `Decoder` class looks like this:

```
#ifndef DECODER_HPP
#define DECODER_HPP

#include <fstream>
#include <vector>
#include <utility>
#include <bitset>

#include "Types.hpp"
#include "Image.hpp"
#include "HuffmanTree.hpp"
#include "MCU.hpp"

namespace kpeg
{
    class Decoder
```

```

{
    public:

        enum ResultCode
        {
            SUCCESS,
            TERMINATE,
            ERROR,
            DECODE_INCOMPLETE,
            DECODE_DONE
        };

    public:

        Decoder();

        Decoder(const std::string& filename);

        ~Decoder();

        bool open(const std::string& filename);

        ResultCode decodeImageFile();

        bool dumpRawData();

        void close();

    private:

        ResultCode parseSegmentInfo(const UInt8 byte);

        void parseAPP0Segment();

        void parseCOMSegment();

        void parseDQTSegment();

        ResultCode parseSOF0Segment();

        void parseDHTSegment();

        void parseSOSSegment();

        void scanImageData();

        void byteStuffScanData();

        void decodeScanData();

    private:

        std::string m_filename;
        std::ifstream m_imageFile;
        Image m_image;
        std::vector<std::vector<UInt16>> m_QTables;
        HuffmanTable m_huffmanTable[2][2];
        std::vector< std::pair<int, int> > mDHTsScanned;
        HuffmanTree m_huffmanTree[2][2];
        std::string m_scanData;
        std::vector<MCU> m_MCU;
    };
}

#endif // DECODER_HPP

```

Implementing the Decoder

Let's implement the public interface for `Decoder` first:

```
#include <arpa/inet.h> // htons
#include <iomanip>
#include <sstream>

#include "Decoder.hpp"
#include "Markers.hpp"
#include "Utility.hpp"

namespace kpeg
{

Decoder::Decoder()
{

    logFile << "Created \'Decoder object\'.\" << std::endl;
}

Decoder::Decoder(const std::string& filename)
{
    logFile << "Created \'Decoder object\'.\" << std::endl;
}

Decoder::~Decoder()
{
    close();
    logFile << "Destroyed \'Decoder object\'.\" << std::endl;
}

bool Decoder::open(const std::string& filename)
{
    m_imageFile.open(filename, std::ios::in | std::ios::binary);

    if (!m_imageFile.is_open() || !m_imageFile.good())
    {
        logFile << "Unable to open image: \'\" + filename + "\'\"
    << std::endl;
        return false;
    }

    logFile << "Opened JPEG image: \'\" + filename + "\'\" << st
d::endl;

    m_filename = filename;

    return true;
}

Decoder::ResultCode Decoder::decodeImageFile()
{
    if (!m_imageFile.is_open() || !m_imageFile.good())
    {
        logFile << "Unable scan image file: \'\" + m_filename +
    "\'\" << std::endl;
        return ResultCode::ERROR;
    }

    logFile << "Started decoding process...\" << std::endl;

    UInt8 byte;
    ResultCode status = ResultCode::DECODE_DONE;
```

```

while (m_imageFile >> std::noskipws >> byte)
{
    if (byte == JFIF_BYTE_FF)
    {
        m_imageFile >> std::noskipws >> byte;

        ResultCode code = parseSegmentInfo(byte);

        if (code == ResultCode::SUCCESS)
            continue;
        else if (code == ResultCode::TERMINATE)
        {
            status = ResultCode::TERMINATE;
            break;
        }
        else if (code == ResultCode::DECODE_INCOMPLETE)
        {
            status = ResultCode::DECODE_INCOMPLETE;
            break;
        }
    }
    else
    {
        logFile << "[ FATAL ] Invalid JFIF file! Terminatin
g..." << std::endl;
        status = ResultCode::ERROR;
        break;
    }
}

if (status == ResultCode::DECODE_DONE)
{
    decodeScanData();
    m_image.createImageFromMCUs(m_MCU);
    logFile << "Finished decoding process [OK]." << std::end
1;
}
else if (status == ResultCode::TERMINATE)
{
    logFile << "Terminated decoding process [NOT-OK]." << st
d::endl;
}
else if (status == ResultCode::DECODE_INCOMPLETE)
{
    logFile << "Decoding process incomplete [NOT-OK]." << st
d::endl;
}

return status;
}

bool Decoder::dumpRawData()
{
    std::size_t extPos = m_filename.find(".jpg");

    if (extPos == std::string::npos)
        extPos = m_filename.find(".jpeg");

    std::string targetFilename = m_filename.substr(0, extPos) +
".ppm";
    m_image.dumpRawData(targetFilename);

    return true;
}

```

```

void Decoder::close()
{
    m_imageFile.close();
    logFile << "Closed image file: \"" + m_filename + "\"" << std::endl;
}

}

```

The code for the public interface is pretty self explanatory. The main point of interest is the `decodeImageData` method. What it does is it reads the image file, looking for markers, and when it finds one, it invokes the `parseSegmentInfo` to check whether it's a valid marker, and then this method in turn invokes the appropriate method to handle that marker, e.g., if `decodeImageData` encounters the byte `ff`, it will read the next byte, and if it's a valid marker, say, `fe`, it will invoke the `parseCOMSegment` method to read the comment segment.

parseSegmentInfo

Parsing the segment info is pretty straightforward - check whether the byte after a `ff` is one of the recognized markers corresponding to a segment or not. If yes, we call the appropriate method to parse that segment.

```

Decoder::ResultCode Decoder::parseSegmentInfo(const UInt8 byte)
{
    if (byte == JFIF_BYTE_0 || byte == JFIF_BYTE_FF)
        return ERROR;

    switch(byte)
    {
        case JFIF_SOI : logFile << "Found segment, Start of Image (FFD8)" << std::endl; return ResultCode::SUCCESS;
        case JFIF_APP0 : logFile << "Found segment, JPEG/JFIF Image Marker segment (APP0)" << std::endl; parseAPP0Segment(); return ResultCode::SUCCESS;
        case JFIF_COM : logFile << "Found segment, Comment (FFE)" << std::endl; parseCOMSegment(); return ResultCode::SUCCESS;
        case JFIF_DQT : logFile << "Found segment, Define Quantization Table (FFDB)" << std::endl; parseDQTSegment(); return ResultCode::SUCCESS;
        case JFIF_SOF0 : logFile << "Found segment, Start of Frame 0: Baseline DCT (FFC0)" << std::endl; return parseSOF0Segment();
        case JFIF_SOF1 : logFile << "Found segment, Start of Frame 1: Extended Sequential DCT (FFC1), Not supported" << std::endl; return ResultCode::TERMINATE;
        case JFIF_SOF2 : logFile << "Found segment, Start of Frame 2: Progressive DCT (FFC2), Not supported" << std::endl; return ResultCode::TERMINATE;
        case JFIF_SOF3 : logFile << "Found segment, Start of Frame 3: Lossless Sequential (FFC3), Not supported" << std::endl; return ResultCode::TERMINATE;
        case JFIF_SOF5 : logFile << "Found segment, Start of Frame 5: Differential Sequential DCT (FFC5), Not supported" << std::endl; return ResultCode::TERMINATE;
        case JFIF_SOF6 : logFile << "Found segment, Start of Frame 6: Differential Progressive DCT (FFC6), Not supported" << std::endl; return ResultCode::TERMINATE;
    }
}

```

```

d::endl; return ResultCode::TERMINATE;
    case JFIF_SOF7 : logFile << "Found segment, Start of Fra
me 7: Differential lossless (Sequential) (FFC7), Not supported"
<< std::endl; return ResultCode::TERMINATE;
    case JFIF_SOF9 : logFile << "Found segment, Start of Fra
me 9: Extended Sequential DCT, Arithmetic Coding (FFC9), Not sup
ported" << std::endl; return ResultCode::TERMINATE;
    case JFIF_SOF10: logFile << "Found segment, Start of Fra
me 10: Progressive DCT, Arithmetic Coding (FFCA), Not supported"
<< std::endl; return ResultCode::TERMINATE;
    case JFIF_SOF11: logFile << "Found segment, Start of Fra
me 11: Lossless (Sequential), Arithmetic Coding (FFCB), Not supp
orted" << std::endl; return ResultCode::TERMINATE;
    case JFIF_SOF13: logFile << "Found segment, Start of Fra
me 13: Differential Sequential DCT, Arithmetic Coding (FFCD), N
ot supported" << std::endl; return ResultCode::TERMINATE;
    case JFIF_SOF14: logFile << "Found segment, Start of Fra
me 14: Differential Progressive DCT, Arithmetic Coding (FFCE),
Not supported" << std::endl; return ResultCode::TERMINATE;
    case JFIF_SOF15: logFile << "Found segment, Start of Fra
me 15: Differential Lossless (Sequential), Arithmetic Coding (F
FCF), Not supported" << std::endl; return ResultCode::TERMINATE;
    case JFIF_DHT : logFile << "Found segment, Define Huff
man Table (FFC4)" << std::endl; parseDHTSegment(); return Result
Code::SUCCESS;
    case JFIF_SOS : logFile << "Found segment, Start of Sc
an (FFDA)" << std::endl; parseSOSSegment(); return ResultCode::S
UCCESS;
    }

    return ResultCode::SUCCESS;
}

```

parseAPP0Segment

This method is invoked when the APP-0 segment marker is encountered. Here we parse the identifier “JFIF\0”, major & minor version numbers of the JFIF file pixel unit density (X and Y units ratio 1:1, pixels per inch or pixels per cm), horizontal & vertical pixel density. We ignore the thumbnail data, if present.

```

void Decoder::parseAPP0Segment()
{
    if (!m_imageFile.is_open() || !m_imageFile.good())
    {
        logFile << "Unable scan image file: \' + m_filename +
\'" << std::endl;
        return;
    }

    logFile << "Parsing JPEG/JFIF marker segment (APP-0)..." <<
std::endl;

    UInt16 lenByte = 0;
    UInt8 byte = 0;

    m_imageFile.read(reinterpret_cast<char *>(&lenByte), 2);
    lenByte = htons(lenByte);
    std::size_t curPos = m_imageFile.tellg();

    logFile << "JFIF Application marker segment length: " << len
Byte << std::endl;
}

```



```

// Skip the 'JFIF\0' bytes
m_imageFile.seekg(5, std::ios_base::cur);

UInt8 majVersionByte, minVersionByte;
m_imageFile >> std::noskipws >> majVersionByte >> minVersion
Byte;

logFile << "JFIF version: " << (int)majVersionByte << "." <<
(int)(minVersionByte >> 4) << (int)(minVersionByte & 0x0F) << st
d::endl;

std::string majorVersion = std::to_string(majVersionByte);
std::string minorVersion = std::to_string((int)(minVersionBy
te >> 4));
minorVersion += std::to_string((int)(minVersionByte & 0x0
F));

UInt8 densityByte;
m_imageFile >> std::noskipws >> densityByte;

std::string densityUnit = "";
switch(densityByte)
{
    case 0x00: densityUnit = "Pixel Aspect Ratio"; break;
    case 0x01: densityUnit = "Pixels per inch (DPI)"; break;
    case 0x02: densityUnit = "Pixels per centimeter"; break;
}

logFile << "Image density unit: " << densityUnit << std::end
l;

UInt16 xDensity = 0, yDensity = 0;

m_imageFile.read(reinterpret_cast<char *>(&xDensity), 2);
m_imageFile.read(reinterpret_cast<char *>(&yDensity), 2);

xDensity = htons(xDensity);
yDensity = htons(yDensity);

logFile << "Horizontal image density: " << xDensity << std::
endl;
logFile << "Vertical image density: " << yDensity << std::en
dl;

// Ignore the image thumbnail data
UInt8 xThumb = 0, yThumb = 0;
m_imageFile >> std::noskipws >> xThumb >> yThumb;
m_imageFile.seekg(3 * xThumb * yThumb, std::ios_base::cur);

logFile << "Finished parsing JPEG/JFIF marker segment (APP-
0) [OK]" << std::endl;
}

```

parseCOMSegment

Here we parse the comment, if one is present, in the JFIF file.

```

void Decoder::parseCOMSegment()
{
    if (!m_imageFile.is_open() || !m_imageFile.good())
    {
        logFile << "Unable scan image file: \" + m_filename +
        "\"" << std::endl;
        return;
    }
}

```

```

    }

    logFile << "Parsing comment segment..." << std::endl;

    UInt16 lenByte = 0;
    UInt8 byte = 0;
    std::string comment;

    m_imageFile.read(reinterpret_cast<char*>(&lenByte), 2);
    lenByte = htons(lenByte);
    std::size_t curPos = m_imageFile.tellg();

    logFile << "Comment segment length: " << lenByte << std::endl;

    for (auto i = 0; i < lenByte - 2; ++i)
    {
        m_imageFile >> std::noskipws >> byte;

        if (byte == JFIF_BYTE_FF)
        {
            logFile << "Unexpected start of marker at offset: "
            << curPos + i << std::endl;
            logFile << "Comment segment content: " << comment <<
            std::endl;
            return;
        }

        comment.push_back(static_cast<char>(byte));
    }

    logFile << "Comment segment content: " << comment << std::endl;
    logFile << "Finished parsing comment segment [OK]" << std::endl;
}

```

parseDQTSegment

This method parses one or more quantization tables present in a DQT segment.

```

void Decoder::parseDQTSegment()
{
    if (!m_imageFile.is_open() || !m_imageFile.good())
    {
        logFile << "Unable scan image file: \"" + m_filename +
        "\"\" << std::endl;
        return;
    }

    logFile << "Parsing quantization table segment..." << std::endl;

    UInt16 lenByte = 0;
    UInt8 PqTq;
    UInt8 Qi;

    m_imageFile.read(reinterpret_cast<char*>(&lenByte), 2);
    lenByte = htons(lenByte);
    logFile << "Quantization table segment length: " << (int)len
    Byte << std::endl;

    lenByte -= 2;

```

```

for (int qt = 0; qt < int(lenByte) / 65; ++qt)
{
    m_imageFile >> std::noskipws >> PqTq;

    int precision = PqTq >> 4; // Precision is always 8-bit
    for baseline DCT
    int QTtable = PqTq & 0x0F; // Quantization table number
    (0-3)

    logFile << "Quantization Table Number: " << QTtable << s
    td::endl;
    logFile << "Quantization Table #" << QTtable << " precis
    ion: " << (precision == 0 ? "8-bit" : "16-bit") << std::endl;

    m_QTables.push_back({});

    // Populate quantization table #QTtable
    for (auto i = 0; i < 64; ++i)
    {
        m_imageFile >> std::noskipws >> Qi;
        m_QTables[QTtable].push_back((UInt16)Qi);
    }
}

logFile << "Finished parsing quantization table segment [0
K]" << std::endl;
}

```

parseSOF0Segment

This method parses the Frame-0 segment (SOF-0), which is meant for baseline, DCT JPEG.

```

Decoder::ResultCode Decoder::parseSOF0Segment()
{
    if (!m_imageFile.is_open() || !m_imageFile.good())
    {
        logFile << "Unable scan image file: \"" + m_filename +
        "\" " << std::endl;
        return ResultCode::ERROR;
    }

    logFile << "Parsing SOF-0 segment..." << std::endl;

    UInt16 lenByte, imgHeight, imgWidth;
    UInt8 precision, compCount;

    m_imageFile.read(reinterpret_cast<char *>(&lenByte), 2);
    lenByte = htons(lenByte);

    logFile << "SOF-0 segment length: " << (int)lenByte << std::
    endl;

    m_imageFile >> std::noskipws >> precision;
    logFile << "SOF-0 segment data precision: " << (int)precisio
    n << std::endl;

    m_imageFile.read(reinterpret_cast<char *>(&imgHeight), 2);
    m_imageFile.read(reinterpret_cast<char *>(&imgWidth), 2);

    imgHeight = htons(imgHeight);
    imgWidth = htons(imgWidth);

```

```

logFile << "Image height: " << (int)imgHeight << std::endl;
logFile << "Image width: " << (int)imgWidth << std::endl;

m_imageFile >> std::noskipws >> compCount;

logFile << "No. of components: " << (int)compCount << std::e
ndl;

UInt8 compID = 0, sampFactor = 0, QTNo = 0;

bool isNonSampled = true;

for (auto i = 0; i < 3; ++i)
{
    m_imageFile >> std::noskipws >> compID >> sampFactor >>
QTNo;

    logFile << "Component ID: " << (int)compID << std::endl;
    logFile << "Sampling Factor, Horizontal: " << int(sampFa
ctor >> 4) << ", Vertical: " << int(sampFactor & 0x0F) << std::e
ndl;
    logFile << "Quantization table no.: " << (int)QTNo << st
d::endl;

    if ((sampFactor >> 4) != 1 || (sampFactor & 0x0F) != 1)
        isNonSampled = false;
}

if (!isNonSampled)
{
    logFile << "Chroma subsampling not yet supported!" << st
d::endl;
    logFile << "Chroma subsampling is not 4:4:4, terminatin
g..." << std::endl;
    return ResultCode::TERMINATE;
}

logFile << "Finished parsing SOF-0 segment [OK]" << std::end
l;
m_image.width = imgWidth;
m_image.height = imgHeight;

return ResultCode::SUCCESS;
}

```

parseDHTSegment

Parse the Huffman tables defined in the DHT segment.

```

void Decoder::parseDHTSegment()
{
    if (!m_imageFile.is_open() || !m_imageFile.good())
    {
        logFile << "Unable scan image file: '\" + m_filename +
        \"'\" << std::endl;
        return;
    }

    logFile << "Parsing Huffman table segment..." << std::endl;

    UInt16 len;
    m_imageFile.read(reinterpret_cast<char *>(&len), 2);
    len = htons(len);

```

```

    logFile << "Huffman table length: " << (int)len << std::endl;
1;

    int segmentEnd = (int)m_imageFile.tellg() + len - 2;

    while (m_imageFile.tellg() < segmentEnd)
    {
        UInt8 htinfo;
        m_imageFile >> std::noskipws >> htinfo;

        int HTType = int((htinfo & 0x10) >> 4);
        int HTNumber = int(htinfo & 0x0F);

        logFile << "Huffman table type: " << HTType << std::endl;
1;

        logFile << "Huffman table #: " << HTNumber << std::endl;

        int totalSymbolCount = 0;
        UInt8 symbolCount;

        for (auto i = 1; i <= 16; ++i)
        {
            m_imageFile >> std::noskipws >> symbolCount;
            m_huffmanTable[HTType][HTNumber][i-1].first = (int)s
symbolCount;
            totalSymbolCount += (int)symbolCount;
        }

        // Load the symbols
        int syms = 0;
        for (auto i = 0; syms < totalSymbolCount; )
        {
            // Read the next symbol, and add it to the
            // proper slot in the Huffman table.
            //
            // Depending upon the symbol count, say n, for the
current
            // symbol length, insert the next n symbols in the s
symbol
            // list to it's proper spot in the Huffman table. Th
is means,
            // if symbol counts for symbols of lengths 1, 2 and
3 are 0,
            // 5 and 2 respectively, the symbol list will contai
n 7
            // symbols, out of which the first 5 are symbols wit
h length
            // 2, and the remaining 2 are of length 3.
            UInt8 code;
            m_imageFile >> std::noskipws >> code;

            if (m_huffmanTable[HTType][HTNumber][i].first == 0)
            {
                while (m_huffmanTable[HTType][HTNumber][++i].fir
st == 0);
            }

            m_huffmanTable[HTType][HTNumber][i].second.push_back
(code);
            syms++;

            if (m_huffmanTable[HTType][HTNumber][i].first == m_h
uffmanTable[HTType][HTNumber][i].second.size())
                i++;
        }

        logFile << "Printing symbols for Huffman table (" << HTT

```

```

type << ", " << HTNumber << ")..." << std::endl;

    int totalCodes = 0;
    for (auto i = 0; i < 16; ++i)
    {
        std::string codeStr = "";
        for (auto&& symbol : m_huffmanTable[HTType][HTNumber][i].second)
        {
            std::stringstream ss;
            ss << "0x" << std::hex << std::setfill('0') << symbol;
            std::setw(2) << std::setprecision(16) << (int)symbol;
            codeStr += ss.str() + " ";
            totalCodes++;
        }

        logFile << "Code length: " << i+1
                << ", Symbol count: " << m_huffmanTable[HTType][HTNumber][i].second.size()
                << ", Symbols: " << codeStr
        << std::endl;
    }

    logFile << "Total Huffman codes for Huffman table (Type:"
    << HTType << ", #: " << HTNumber << "): " << totalCodes << std::endl;

    m_huffmanTree[HTType][HTNumber].constructHuffmanTree(m_huffmanTable[HTType][HTNumber]);
    auto htree = m_huffmanTree[HTType][HTNumber].getTree();
    logFile << "Huffman codes:-" << std::endl;
    inOrder(htree);
}

logFile << "Finished parsing Huffman table segment [OK]" <<
std::endl;
}

```

parseSOSSegment

Parse the SOS segment. This segment has information describing which component will use which Huffman table. If we decode a component using the wrong Huffman tree, decoding will either fail or produce utter gibberish! So, we need to pay special attention to make sure we use the appropriate Huffman table for a component while decoding it. This information, though present in the SOS file, is also hardcoded in the `Types` module.

```

void Decoder::parseSOSSegment()
{
    if (!m_imageFile.is_open() || !m_imageFile.good())
    {
        logFile << "Unable scan image file: \"" + m_filename +
        "\"\n" << std::endl;
        return;
    }

    logFile << "Parsing SOS segment..." << std::endl;

    UInt16 len;

```

```

m_imageFile.read(reinterpret_cast<char*>(&len), 2);
len = htons(len);

logFile << "SOS segment length: " << len << std::endl;

UInt8 compCount; // Number of components
UInt16 compInfo; // Component ID and Huffman table used

m_imageFile >> std::noskipws >> compCount;

if (compCount < 1 || compCount > 4)
{
    logFile << "Invalid component count in image scan: " <<
(int)compCount << ", terminating decoding process..." << std::en
dl;
    return;
}

logFile << "Number of components in scan data: " << (int)com
pCount << std::endl;

for (auto i = 0; i < compCount; ++i)
{
    m_imageFile.read(reinterpret_cast<char*>(&compInfo),
2);
    compInfo = htons(compInfo);

    UInt8 cID = compInfo >> 8; // 1st byte denotes component
ID

    // 2nd byte denotes the Huffman table used:
    // Bits 7 to 4: DC Table #(0 to 3)
    // Bits 3 to 0: AC Table #(0 to 3)
    UInt8 DCTableNum = (compInfo & 0x00f0) >> 4;
    UInt8 ACTableNum = (compInfo & 0x000f);

    logFile << "Component ID: " << (int)cID << ", DC Table
#: " << (int)DCTableNum << ", AC Table #: " << (int)ACTableNum <
< std::endl;
}

// Skip the next three bytes
for (auto i = 0; i < 3; ++i)
{
    UInt8 byte;
    m_imageFile >> std::noskipws >> byte;
}

logFile << "Finished parsing SOS segment [OK]" << std::endl;

scanImageData();
}

```

scanImageData

This is the method that reads the actual image data, i.e., compressed stream of bytes. These bytes can be deciphered from the info obtained from the APP-0, SOF-0, DQT and DHT segments.

```

void Decoder::scanImageData()
{

```

```

        if (!m_imageFile.is_open() || !m_imageFile.good())
        {
            logFile << "Unable scan image file: \'' + m_filename +
            "\'" << std::endl;
            return;
        }

        logFile << "Scanning image data..." << std::endl;

        UInt8 byte;

        while (m_imageFile >> std::noskipws >> byte)
        {
            if (byte == JFIF_BYTE_FF)
            {
                UInt8 prevByte = byte;

                m_imageFile >> std::noskipws >> byte;

                if (byte == JFIF_EOI)
                {
                    logFile << "Found segment, End of Image (FFD9)"
                    << std::endl;
                    return;
                }

                std::bitset<8> bits1(prevByte);
                logFile << "0x" << std::hex << std::setfill('0') <<
                std::setw(2) << std::setprecision(8)
                << (int)prevByte << ", Bits: " << bits1 <
                < std::endl;

                m_scanData.append(bits1.to_string());
            }

            std::bitset<8> bits(byte);
            logFile << "0x" << std::hex << std::setfill('0') << st
            d::setw(2) << std::setprecision(8) <<
            (int)byte << ", Bits: " << bits << st
            d::endl;

            m_scanData.append(bits.to_string());
        }

        logFile << "Finished scanning image data [OK]" << std::endl;
    }

```

byteStuffScanData

Since the byte `ff` has a special meaning (MSB of markers), to be able to have the byte `ff` in case it occurs in the scan data, a process known as byte stuffing is performed while encoding. For example, If the bytes `ff d8` were to occur in the scan data (which is the marker for SOI), a mechanism is needed to avoid misinterpreting it as the SOI segment. Byte stuffing involves appending an empty byte (`00`) to any `ff` bytes that occur in the scan data. So, in our example `ff d8` would get written to the JFIF file as `ff 00 d8`.

Therefore, we need to remove the occurrences of `00` after `ff` in the scan data before we can decode it. This method does exactly that.

```
void Decoder::byteStuffScanData()
{
    if (m_scanData.empty())
    {
        logFile << " [ FATAL ] Invalid image scan data" << std::endl;
        return;
    }

    logFile << "Byte stuffing image scan data..." << std::endl;

    for (unsigned i = 0; i <= m_scanData.size() - 8; i += 8)
    {
        std::string byte = m_scanData.substr(i, 8);

        if (byte == "11111111")
        {
            if (i + 8 < m_scanData.size() - 8)
            {
                std::string nextByte = m_scanData.substr(i + 8, 8);

                if (nextByte == "00000000")
                {
                    m_scanData.erase(i + 8, 8);
                }
                else continue;
            }
            else
                continue;
        }
    }

    logFile << "Finished byte stuffing image scan data [OK]" << std::endl;
}
```

decodeScanData

This is perhaps the most complex method in the decoder, which decodes the scan data obtained. Here's how the decode logic works (this assumes that byte stuffing has been reversed):

```
1.  MCU-Count := (image-width * image-height) / 64
2.  MCU-List := Empty list of MCUs
3.  k := 0 // The index of the next bit to read from scan data
4.  FOR i in range (1, MCU-Count) DO
5.      RLE := Empty 2D vector of dimensions (3 x n)
6.      FOR i in range(1..3) DO // there are three components i
n Y-Cb-Cr
7.          bitsScanned := "" // Initially no bits have been scanned
8.          WHILE TRUE DO // Keep reading bits until encounter
a legit Huffman code for the DC coefficient for component-i
9.              nextBit := Read next bit in scan data
```

```

10.             bitsScanned.Append(nextBit)
11.             value := Get symbol corresponding to bitsScanne
d from HT for component-i, DC
12.             IF value is a valid symbol corresponding to Huf
fman code in bitsScanne THEN
13.                 IF value EQUALS EOB THEN
14.                     k := k + 1
15.                     RLE[i].PUSH(0) // (0,0) is the RLE code
corresponding to EOB
16.                     RLE[i].PUSH(0)
17.                 ELSE
18.                     zeroCount := value >> 4 // Get the first
four MSBs of value, i.e., bits 7..4
19.                     category := value & 0x0f // Get the last
four LSBs of value, i.e., bits 3..0
20.                     coeffBits := Read `category` bits starti
ng from (k+1)
21.                     DCCoeff := bitStringToValue(coeffBits)
22.                     k := k + (category + 1) // Category of a
value is the # of bits in its bit string representation
23.                     RLE[i].PUSH(zeroCount)
24.                     RLE[i].PUSH(DCCoeff)
25.                 END-IF
26.                 bitsScanned := ""
27.                 BREAK
28.             END-IF
29.             k := k + 1
30.         END-WHILE
31.         AC-code-count := 0
32.         WHILE TRUE DO // Keep reading bits until encounter a
legit Huffman code for the AC coefficient for component-i
33.             IF AC-code-count EQUALS 63 THEN
34.                 BREAK
35.             END-IF
36.             nextBit := Read next bit in scan data
37.             bitsScanned.Append(nextBit)
38.             value := Get symbol corresponding to bitsScanned
from HT for component-i, AC
39.             IF value is a valid symbol corresponding to Huff
man code in bitsScanne THEN
40.                 IF value EQUALS EOB THEN
41.                     k := k + 1
42.                     RLE[i].PUSH(0) // (0,0) is the RLE code
corresponding to EOB
43.                     RLE[i].PUSH(0)
44.                     BREAK
45.                 ELSE
46.                     zeroCount := value >> 4 // Get the first
four MSBs of value, i.e., bits 7..4
47.                     category := value & 0x0f // Get the last
four LSBs of value, i.e., bits 3..0
48.                     coeffBits := Read `category` bits starti
ng from (k+1)
49.                     ACCoeff := bitStringToValue(coeffBits)
50.                     k := k + (category + 1) // Category of a
value is the # of bits in its bit string representation
51.                     RLE[i].PUSH(zeroCount)
52.                     RLE[i].PUSH(DCCoeff)
53.                     AC-code-count := AC-code-count + zerCoun
t + 1
54.                 END-IF
55.                 bitsScanned := ""
56.             END-IF
57.             k := k + 1
58.         WND-WHILE
59.         IF RLE[i].length EQUALS 2 THEN // both the DC and AC
coefficients are EOB

```

```

60.             RLE[i].POP() //Remove the extra EOB, (0, 0), from
the RLE for component-i
61.             RLE[i].POP()
62.         END-IF
63.     END-FOR
64.     new-MCU = CreateMCU(RLE, QT) // QT are the quantization
tables read from DQT segments previously
65.     MCU-List.PUSH(new-MCU)
66. END-FOR

```

I know that the above pseudocode may not even make sense just by looking at it. Hence, let's take a concrete example. Remember the Huffman encoded scan data from part 1? Let's use that as the scan data for our example. This data is for a simple component (Y) and single MCU and was encoded using the standard DC (ITU-T.81, Annex-K, Section-K.3.1, Table-K.3, Page- 149) and AC (ITU-T.81, Annex-K, Section-K.3.2, Table-K.5, Page- 150) Huffman tables from the JPEG specification. Also, we will refer to the [value category & bitstream table](#) listed in part 1.

1100000101101000101110010100101011101111110101111010110101010

So, first, we start decoding the DC coefficient. We keep reading the bits until we encounter a Huffman code for DC:

110

00001011010001011100101001010111011111101011110101101010

Consulting the table for DC table for Y , we see that the Huffman code 110 corresponds to the symbol 5, which denotes the value category for the DC coefficient. Reading the next 5 bits in the scan data, we get:

110 00001

011010001011100101001010111011111101011110101101010

Looking up the bitstream 00001 under category 5 in the value category & bitstream table, we see 00001 corresponds to -31 .

Now, let's move on to the AC coefficients. We keep reading the scan data until we get a valid Huffman code for the AC table.

110 00001 01

1010001011100101001010111011111101011110101101010

Looking up the code 01 on the Huffman table for AC coefficients, we see that it corresponds to the zero run-length & value category (0 2), which means that the zero run-length, $RRRR = 0$, and the value category $SSSS = 2$. Reading the next two bits in the scan data, we get:

110 00001 01 10

10001011100101001010111011111101011110101101010

Looking up **10** in the value category & bitsream table, we see that it represents **2**. So, the run-length encoded data is **(0, 2)**.

Let's decode one more AC coefficient. We read determine the next valid Huffman code in the scan data from the AC Huffman table:

110 00001 01 10 100
0101110010100101011101111101011110101101010

the Huffman code **100** corresponds to the zero run-length & value category **(0 3)**, where $RRRR = 0$ and $SSSS = 3$. Hence, we read the next three bits:

110 00001 01 10 100 010
1110010100101011101111101011110101101010

Looking up **010** on the value category & bitstream table, we see that it represents **-5**. So, the run-length encoded data is **(0, -5)**.

Proceeding this way, all the remaining AC coefficients can be decoded back to it's run-length coding to obtain the uncompressed data for the MCU. Once that is done, we already have defined our **MCU** class to convert its raw, uncompressed form in the RGB colourspace.

With all that explanation done, here's the implementation for `decodeScanData` :

```
void Decoder::decodeScanData()
{
    if (m_scanData.empty())
    {
        logFile << " [ FATAL ] Invalid image scan data" << std::endl;
        return;
    }

    byteStuffScanData();

    logFile << "Decoding image scan data..." << std::endl;

    const char* component[] = { "Y (Luminance)", "Cb (Chrominance)", "Cr (Chrominance)" };
    const char* type[] = { "DC", "AC" };

    int MCUCount = (m_image.width * m_image.height) / 64;

    m_MCU.clear();
    logFile << "MCU count: " << MCUCount << std::endl;

    int k = 0; // The index of the next bit to be scanned

    for (auto i = 0; i < MCUCount; ++i)
    {
        logFile << "Decoding MCU-" << i + 1 << "..." << std::endl;
    }
}
```

```

// The run-length coding after decoding the Huffman data
std::array<std::vector<int>, 3> RLE;

// For each component Y, Cb & Cr, decode 1 DC
// coefficient and then decode 63 AC coefficients.
//
// NOTE:
// Since a significant portion of a RLE for a
// component contains a trail of 0s, AC coefficients
// are decoded till, either an EOB (End of block) is
// encountered or 63 AC coefficients have been decoded.

for (auto compID = 0; compID < 3; ++compID)
{
    std::string bitsScanned = ""; // Initially no bits are scanned

    // Firstly, decode the DC coefficient
    logFile << "Decoding MCU-" << i + 1 << ": " << component[compID] << "/" << type[HT_DC] << std::endl;

    int HuffTableID = compID == 0 ? 0 : 1;

    while (1)
    {
        bitsScanned += m_scanData[k];
        auto value = m_huffmanTree[HT_DC][HuffTableID].contains(bitsScanned);

        if (!utils::isStringWhiteSpace(value))
        {
            if (value != "EOB")
            {
                int zeroCount = UInt8(std::stoi(value)) >> 4;

                int category = UInt8(std::stoi(value)) & 0x0F;

                int DCCoeff = bitStringtoValue(m_scanData.substr(k + 1, category));

                k += category + 1;
                bitsScanned = "";

                RLE[compID].push_back(zeroCount);
                RLE[compID].push_back(DCCoeff);

                break;
            }

            else
            {
                bitsScanned = "";
                k++;

                RLE[compID].push_back(0);
                RLE[compID].push_back(0);

                break;
            }
        }
        else
        {
            k++;
        }
    }

    // Then decode the AC coefficients
    logFile << "Decoding MCU-" << i + 1 << ": " << component[compID] << "/" << type[HT_AC] << std::endl;

```

```

        bitsScanned = "";
        int ACCodesCount = 0;

        while (1)
        {
            // If 63 AC codes have been encountered, this block is done, move onto next block
            if (ACCodesCount == 63)
            {
                break;
            }

            // Append the k-th bit to the bits scanned so far
            bitsScanned += m_scanData[k];
            auto value = m_huffmanTree[HT_AC][HuffTableID].contains(bitsScanned);

            if (!utils::isStringWhiteSpace(value))
            {
                if (value != "EOB")
                {
                    int zeroCount = UInt8(std::stoi(value)) >> 4;
                    int category = UInt8(std::stoi(value)) & 0x0F;
                    int ACCoeff = bitStringtoValue(m_scanData.substr(k + 1, category));

                    k += category + 1;
                    bitsScanned = "";

                    RLE[compID].push_back(zeroCount);
                    RLE[compID].push_back(ACCoeff);

                    ACCodesCount += zeroCount + 1;
                }
                else
                {
                    bitsScanned = "";
                    k++;

                    RLE[compID].push_back(0);
                    RLE[compID].push_back(0);

                    break;
                }
            }
            else
            {
                k++;
            }
        }

        // If both the DC and AC coefficients are EOB, truncate to (0,0)
        if (RLE[compID].size() == 2)
        {
            bool allZeros = true;

            for (auto&& rVal : RLE[compID])
            {
                if (rVal != 0)
                {
                    allZeros = false;
                    break;
                }
            }
        }
    }
}

```

```

    }

    // Remove the extra (0,0) pair
    if (allZeros)
    {
        RLE[compID].pop_back();
        RLE[compID].pop_back();
    }
}

// Construct the MCU block from the RLE &
// quantization tables to a 8x8 matrix
m_MCU.push_back(MCU(RLE, m_QTables));

logFile << "Finished decoding MCU-" << i + 1 << " [OK]"
<< std::endl;
}

// The remaining bits, if any, in the scan data are discarded as
// they're added byte align the scan data.

logFile << "Finished decoding image scan data [OK]" << std::endl;
}

```

Putting it all Together

Hurrah! All the required components are in place for us to integrate them and take our decoder for a test run! In the `main.cpp` file in the project, we will define a CLI to use our decoder.

```

#include <cmath>

#include "Utility.hpp"
#include "Decoder.hpp"

void printHelp()
{
    std::cout << "===== " <<
    < std::endl;
    std::cout << "    K-PEG - Simple JPEG Encoder & Decoder" <<
    < std::endl;
    std::cout << "===== " <<
    < std::endl;
    std::cout << "Help\n" << std::endl;
    std::cout << "<filename.jpg>                : Decompress a
JPEG image to a PPM image" << std::endl;
    std::cout << "-h                        : Print this h
elp message and exit" << std::endl;
}

void decodeJPEG(const std::string& filename)
{
    if ( !kpeg::utils::isValidFilename( filename ) )
    {
        std::cout << "Invalid input file name passed." << std::e
        ndl;
    }
}

```

```

        return;
    }

    std::cout << "Decoding..." << std::endl;

    kpeg::Decoder decoder;

    decoder.open( filename );
    if ( decoder.decodeImageFile() == kpeg::Decoder::ResultCode::DECODE_DONE )
    {
        decoder.dumpRawData();
    }

    decoder.close();

    std::cout << "Generated file: " << filename.substr(0, filename.length() - 3 ) << ".ppm" << std::endl;
    std::cout << "Complete! Check log file \'kpeg.log\' for details." << std::endl;
}

int handleInput(int argc, char** argv)
{
    if ( argc < 2 )
    {
        std::cout << "No arguments provided." << std::endl;
        return EXIT_FAILURE;
    }

    if ( argc == 2 && (std::string)argv[1] == "-h" )
    {
        printHelp();
        return EXIT_SUCCESS;
    }
    else if ( argc == 2 )
    {
        decodeJPEG( argv[1] );
        return EXIT_SUCCESS;
    }

    std::cout << "Incorrect usage, use -h to view help" << std::endl;
    return EXIT_FAILURE;
}

int main( int argc, char** argv )
{
    try
    {
        logFile << "libKPEG - A simple JPEG library" << std::endl;

        return handleInput(argc, argv);
    }
    catch( std::exception& e )
    {
        std::cout << "Exceptions Occurred:-" << std::endl;
        std::cout << "What: " << e.what() << std::endl;
    }

    return EXIT_SUCCESS;
}

```

Now, let's build our application. Go to the project directory, `libKPEG` (or whatever you named your project), and run the

CMake build script:

```
$ mkdir build && cd build  
$ cmake ..  
$ make
```

This will build our code to generate a binary executable called `kpeg`.

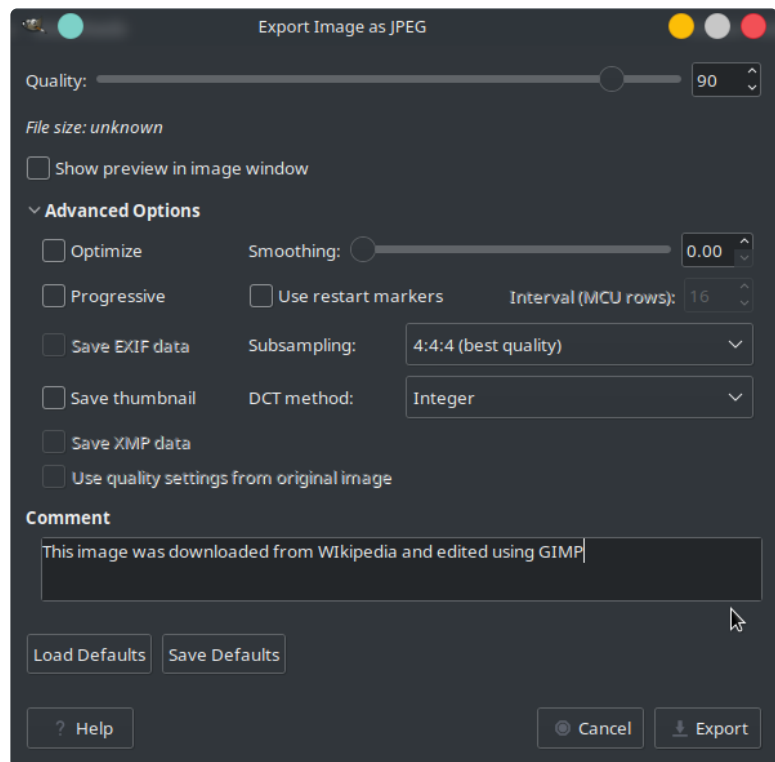
NOTE

If you're not using CMake, you will need to run the build step according to the environment and tools you are using!

Here's the sample image, Lenna, that we're going to use!



This image was borrowed from [Wikipedia](https://en.wikipedia.org/wiki/File:Lenna.png), and converted to sequential, base line DCT, with no chroma subsampling (4:4:4) using GIMP with the following settings:



Save this image in the `build` directory where the `kpeg` binary was created. And now, finally it's time to see all our effort in action! Run `kpeg` on the image:

```
$ ./kpeg lenna.jpg
Decoding...
Generated file: lenna.ppm
Complete! Check log file 'kpeg.log' for details.
```

The command should run with some basic output to the console. Additionally, a detailed log is also generated every run.

Now, the fruit of this article, the decoded JFIF file, written to the disk in the PPM format. You can download the one generated when I ran the decoder [lenna.ppm](#) (can't display PPM files in HTML without some JS-fu!).

Summary

Whew! And you thought that this article would never get over!



Here's what we covered:

- Understanding the JFIF file format, its markers and segments
- Designed high level components of a JPEG decoder
- Wrote the decoder using these components

Now that you know how to write a decoder, writing the encoder shouldn't be much difficult, as an encoder is basically the steps of decoding in reverse order (as described in part 1). I hope to write a post describing an `Encoder` module that will complement `Decoder` and we can call our library complete!

Till then, adios!

References

I acknowledge that I have consulted the following sources while writing this article and the material covered in these resources have helped me understand JPEG:

- The official JPEG standard, ITU-T.81 ([pdf](#))
- The official JFIF standard ([pdf](#))
- Calvin Hass' articles ([blog posts](#))
- Cristi Cuturicu's JPEG Guide ([txt](#))
- Wikipedia's article on JFIF file structure ([article](#))

Related Posts

[Let's Write a Simple JPEG Library, Part-I: The Basics](#) 25 Nov 2017

[Setup Your Personal Site From Scratch Using Linux, Apache And Jekyll](#) 21 Nov 2016

[Migrating Fleptic to a VPS](#) 13 Nov 2016